

## Практична робота 7

**Тема:** Автентифікація та авторизація

**Мета:** набуття практичних навичок реалізації автентифікації та авторизації у Node.js-застосунках шляхом розширення застосунку з лабораторної роботи №6

**Хід роботи:**

**Варіант 5**

### Завдання 1.

Реєстрація користувачів з хешуванням пароля

Розширте API з лаб. 6 модулем реєстрації користувачів із надійним зберіганням паролів.

```
PS D:\fourth_semester\Node\pr7> npm install bcryptjs
npm audit fix

Run `npm audit` for details.
PS D:\fourth_semester\Node\pr7> npm install -D @types/bcryptjs

added 1 package, and audited 452 packages in 1s

79 packages are looking for funding
run `npm fund` for details

5 vulnerabilities (2 moderate, 2 high, 1 critical)
```

Рис. 7.1 – Встановлено пакети bcryptjs

|      |      |                  |        |      |                                                  |      |
|------|------|------------------|--------|------|--------------------------------------------------|------|
|      |      | Ліщинський В. В. |        |      | ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ | Арк. |
|      |      | Петросян А. Р.   |        |      |                                                  | 1    |
| Змн. | Арк. | № докум.         | Підпис | Дата |                                                  |      |

```

src > models > TS user.model.ts > userSchema
1  import { Schema, model, HydratedDocument } from 'mongoose';
2  import bcrypt from 'bcryptjs';
3
4  export interface User {
5    email: string;
6    passwordHash: string;
7    createdAt: Date;
8    updatedAt: Date;
9  }
10
11  export type UserDocument = HydratedDocument<User>;
12
13  const userSchema = new Schema<User>(
14    {
15      email: {
16        type: String,
17        required: [true, 'Email is required'],
18        unique: true,
19        lowercase: true,
20        trim: true,
21        index: true

```

Рис. 7.2 – Модель User

```

src > routes > TS auth.ts > router.post('/register') callback
1  import { Router, Request, Response, NextFunction } from 'express';
2  import { UserModel } from '../models/user.model';
3  import { validate } from '../middleware/validate';
4  import { registerSchema } from '../schemas/auth.schema';
5
6  const router = Router();
7
8  router.post('/register', validate(registerSchema),
9    async (req: Request, res: Response, next: NextFunction) => {
10     try {
11       const { email, password } = req.body;
12
13       const existingUser = await UserModel.findOne({ email });
14
15       if (existingUser) {
16         res.status(409).json({
17           message: 'User with this email already exists',
18         });
19         return;
20       }

```

Рис. 7.3 – Реалізація ендпоінта register

|      |      |                  |        |      |                                                  |      |
|------|------|------------------|--------|------|--------------------------------------------------|------|
|      |      | Ліщинський В. В. |        |      | ДУ «Житомирська політехніка». 24.121.05.000 - ПЗ | Арк. |
|      |      | Петросян А. Р.   |        |      |                                                  | 2    |
| Змн. | Арк. | № докум.         | Підпис | Дата |                                                  |      |

## Завдання 2

Логін з access та refresh токенами

Реалізуйте видачу JWT-токенів та механізм їх оновлення.

```
PS D:\fourth_semester\Node\pr7> npm install jsonwebtoken cookie-parser
npm audit fix

Run `npm audit` for details.
PS D:\fourth_semester\Node\pr7> npm install -D @types/jsonwebtoken @types/cookie-parser

added 3 packages, and audited 471 packages in 988ms

80 packages are looking for funding
run `npm fund` for details
```

Рис 7.4 - Встановлено потрібні пакети

```
PS D:\fourth_semester\Node\pr7> & "$HOME\.fly\bin\flyctl.exe" secrets list
NAME      DIGEST      STATUS
MONGODB_URI 27a8c12111ccb38a Deployed
JWT_SECRET  66651691df4a000a Deployed

PS D:\fourth_semester\Node\pr7>
```

Рис. 7.5 - Added JWT token to fly.io

```
17 export function signAccessToken(payload: JwtPayload) {
18   return jwt.sign(payload, getJwtSecret(), {
19     expiresIn: '15m',
20   });
21 }
22
23 export function signRefreshToken(payload: JwtPayload) {
24   return jwt.sign(payload, getJwtSecret(), {
25     expiresIn: '30d',
26   });
27 }
28
```

Рис. 7.6 - Налаштування “життя” токенів

```

router.post('/login', validate(loginSchema),
  async (req: Request, res: Response, next: NextFunction) => {
    try {
      const { email, password } = req.body;

      const user = await UserModel.findOne({ email });

      if (!user) {
        res.status(401).json({
          message: 'Invalid email or password',
        });
        return;
      }
    }
  }
);

```

Рис. 7.7 – Додано ендпоінт login

```

router.post('/refresh', async (req: Request, res: Response) => {
  const refreshToken = req.cookies?.refresh_token;

  if(!refreshToken) {
    res.status(401).json({
      message: 'Unauthorized',
    });
    return;
  }

  try {
    const refreshToken: any
    const payload = verifyToken(refreshToken);

    setAuthCookies(res, payload.userId);
  }
});

```

Рис. 7.8 – Ендпоінт refresh



```

router.post('/logout', (_req: Request, res: Response) => {
  res.clearCookie('access_token', cookieOptions);
  res.clearCookie('refresh_token', cookieOptions);

  res.status(200).json({
    message: 'Logout successful',
  });
});

```

Рис. 7.9 - Ендпоінт logout

### Завдання 3

Захищені маршрути та авторизація власника

Захистіть CRUD-ендпоінти ресурсу з лаб. 3–4 та додайте перевірку власника.

```

src > middleware > TS requireAuth.ts > requireAuth > payload
1  import { Request, Response, NextFunction } from 'express';
2  import { verifyToken } from '../utils/jwt';
3
4  export function requireAuth(req: Request, res: Response, next: NextFunction) {
5    const accessToken = req.cookies?.access_token;
6
7    if(!accessToken) {
8      res.status(401).json({
9        message: 'Unauthorized',
10      });
11      return;
12    }
13
14    try {
15      const payload = verifyToken(accessToken);

```

Рис. 7.10 – Реалізував middleware requireAuth

```

20  const carSchema = new Schema<Car>({
47    year: {
49      required: [true, 'Year is required'],
50      min: [1920, 'Year must be greater than or equal to 1920'],
51      max: [currentYear, 'Year must be less than or equal to currentYear'],
52      validate: {
53        validator: Number.isInteger,
54        message: 'Year must be an integer',
55      },
56    },
57    ownerId: {
58      type: Schema.Types.ObjectId,
59      ref: 'User',
60      required: [true, 'Owner is required'],
61      index: true,
62    },

```

Рис. 7.11 – Додано ownerId

```

router.delete('/:id', requireAuth, async (req: Request<IdParam>, res: Response) => {
  try {
    if(!req.userId) {
      res.status(401).json({
        message: 'Unauthorized',
      });
      return;
    }

    const car = await getCarById(req.params.id);

```

```

router.patch('/:id', requireAuth, validate(updateCarSchema), async (req, res) => {
  try {
    if (!req.userId) {
      res.status(401).json({
        message: 'Unauthorized',
      });
      return;
    }

    const car = await getCarById(req.params.id);

    if (!car) {
      res.status(404).json({
        message: 'Car not found',
      });
      return;
    }

    const updatedCar = await updateCar(req.params.id, req.body);
    res.status(200).json(updatedCar);
  } catch (error) {
    next(error);
  }
});

router.post('/', requireAuth, validate(createCarSchema), async (req, res) => {
  try {
    if (!req.userId) {
      res.status(401).json({
        message: 'Unauthorized',
      });
      return;
    }

    const newCar = await createCar({ ...req.body, userId: req.userId });
    res.status(201).json(newCar);
  } catch (error) {
    next(error);
  }
});

```

Рис. 7.12 – Додано перевірки та захист до роутів

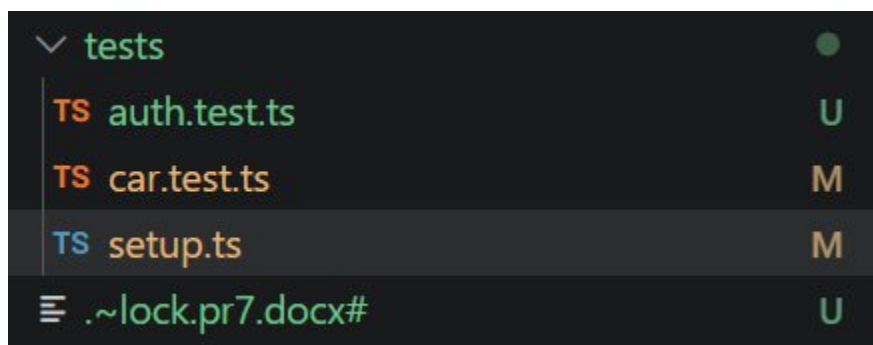


Рис. 7.13 – Додав інтеграційні тести

## Висновок

На даній лабораторній роботі ми набули практичних навичок реалізації автентифікації та авторизації у Node.js-застосунках шляхом розширення застосунку з лабораторної роботи №6